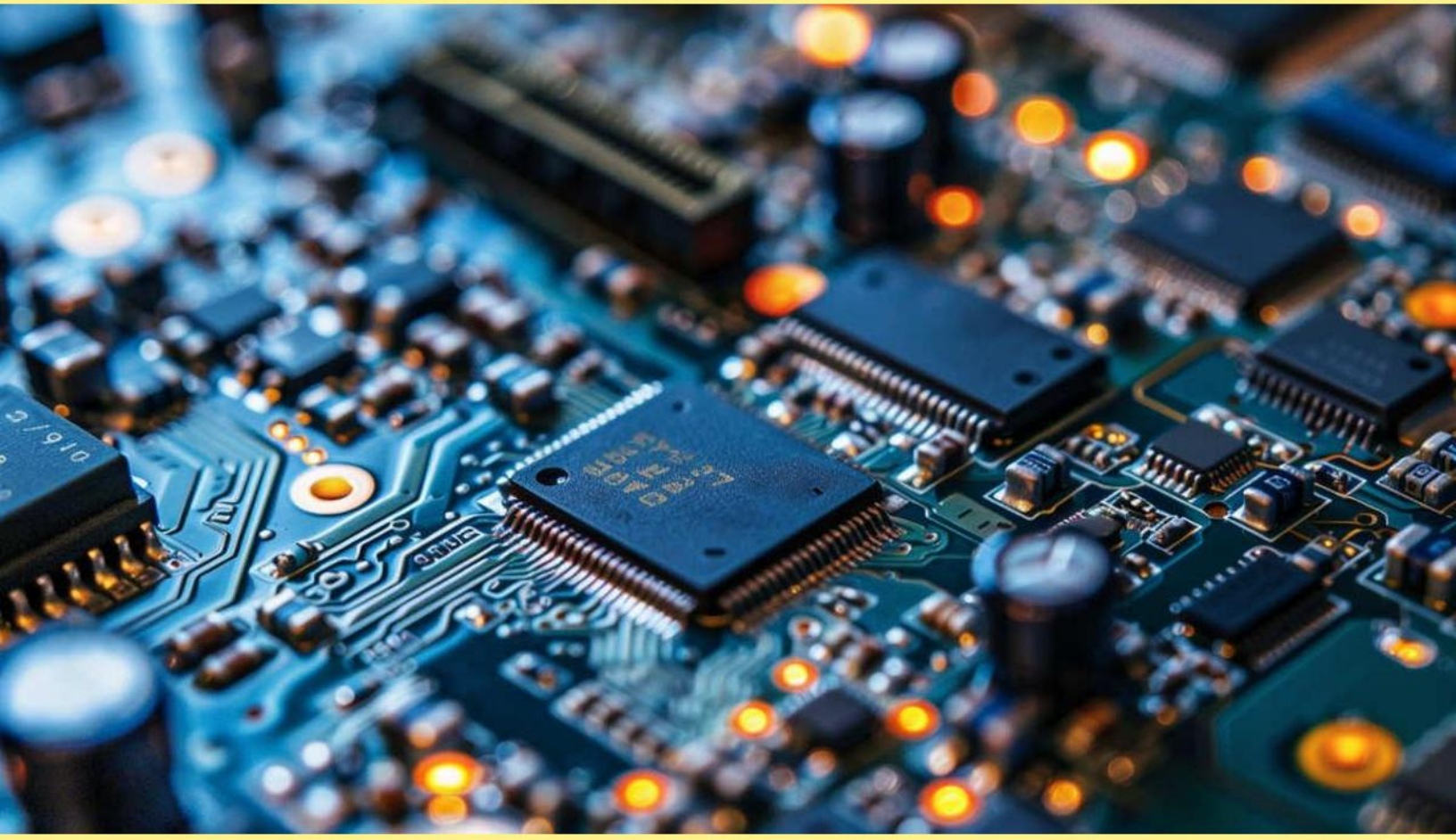
CPI A2 – PROJET SYSTÈMES EMBARQUÉS

PROSIT 4 – ASSEMBLAGE MÉMOIRE



CESI_
ÉCOLE D'INGÉNIEURS

Contexte :

L'équipe MMH commence à avoir un algorithme de gestion des données qui semble convenir à leurs attentes. Il est cependant relativement simpliste. Il nous est demandé

Mots inconnus :

Pointeurs : variable qui contient l'adresse mémoire d'une autre variable, ce qui permet d'accéder à une donnée sans la recopier.
Complexité algorithmique : mesure du nombre d'opérations (complexité en temps) ou de la place mémoire (complexité en espace) nécessaires à un algorithme, en fonction de la taille des données.
Moyenne glissante : moyenne calculée sur les dernières mesures seulement, la plus ancienne étant retirée à chaque nouvelle mesure.
Approximations arithmétiques : remplacement d'un calcul exact par un calcul plus simple et plus rapide (entiers, valeurs tronquées, décalages de bits) dont le résultat reste suffisamment précis.
Data type : type de donnée (int, float, bool…) qui fixe la place occupée en mémoire et les valeurs qu'une variable peut prendre.

Problématique :

Comment optimiser le code de manière à utiliser moins de stockage mémoire et d'effectuer les tâches plus rapidement ?

Plan d'action :

- Étudier les différentes notions inconnues [1h]
- Corriger les potentielles erreurs de format [30 min]
- Évaluer la complexité du programme actuel [25 min]
- Retranscrire le pseudo-code en C [20 min]
- Optimiser la complexité du programme actuel [1h]
- Tester le programme si possible [35 min]

Réalisation :

Pseudo-code de départ

L'équipe MMH a écrit un algorithme de gestion des mesures (fichier `Algo_Mesure.docx`) :

- chaque capteur a une constante (`#define`) qui sert d'indice dans les tableaux ;
- `Tab_Mesures[]` contient la dernière mesure de chaque capteur ;
- `Tab_Erreur[]` signale un échec de mesure ;
- chaque capteur a son propre tableau d'historique pour calculer une moyenne glissante ;
- `Tab_Moy_Instant[]` regroupe les moyennes ;
- les fonctions `Decalage()` et `Calcul_Moy_Instant()` sont communes à tous les tableaux.

Nous sommes partis du pseudo-code suivant :

```
1   void setup():
2
3   #define capteur_fonction = num_capteur (allant de 0 à Nb_Capteurs-1)
4
5   Tab_Mesures[Nb_Capteurs - 1]
6
7   Tab_Erreur[Nb_Capteurs – 1]
8
9   Tab_Moy_Instant[NbCapteurs – 1]
10
11
12  Tab_Capteur_num_capteur[indice_max_Tableau]
13
14
15  void loop() {
16
17  Lecture_Capteurs()
18
19  Maj_Mesures(…,…)
20
21  Calcul_Moy_Instant() pour chaque tableau de capteur
22
23  Les fonctions/procédures principales :
24  PROCEDURE LECTURE_CAPTEURS
25
26  Lecture_Capteurs()
27
28  Tab_Erreur = 0
29
30  Pour chaque capteur (num_capteur) :
31
32                Si Mesure_capteur <> « erreur » alors Tab_Mesures[num_capteur] = Mesure_capteur
    Sinon Tab_erreur[num_capteur] = 1
33
34
35
36
37  PROCEDURE MAJ_MESURES
38
39  int[] Maj_Mesures(Tableau[],Erreur[]): {
40
41
42  Une instruction par tableau de capteur :
43
44  Si Erreur[num_capteur] <> 1 alors
45
46  Tableau_Capteur_num_capteur[] = Decalage(Tableau_Capteur_num_capteur[], Tableau[num_capteur])
47
48
49  int[] Decalage(Tableau[], Valeur): {
50
51  Pour i allant de 0 à indice_max_Tableau - 1
52
53  Tableau[i] = Tableau [i+1]
```

```
54
55  Tableau[indice_max_Tableau] = Valeur
56
57  Return Tableau[]
58
59
60
61  Moy_Tableau Calcul_Moy_Instant(Tableau[])
62
63  Moyenne = 0
64
65  Pour chaque i de Tableau
66
67  Moyenne = Moyenne + Tableau[i]
68
69  Moyenne = Moyenne / (indice_max_Tableau + 1)
70
71  Return Moyenne
```

Retour de la supervision

La supervision a refusé cette version :

1. la complexité doit être réduite, car le temps d'exécution doit être minimal ;
2. les tableaux doivent être remplacés par des structures ;
3. la mémoire est limitée ;
4. les mesures n'ont pas besoin de nombres réels (`float`, qui montent jusqu'à environ $\pm 3{,}4 \cdot 10^{38}$, soit environ 2^{128}), d'autant plus qu'elles sont tronquées pendant le traitement.

La remarque finale (« il leur reste toujours la solution de tester le programme, à moins que… ») indique qu'on n'a pas besoin de tester pour savoir si le programme est efficace : on peut le **démontrer par le calcul de complexité**.

Étapes réalisées

Étape	Contenu
1	Traduction du pseudocode en C++ Arduino (**version 1**)
2	Calcul de la complexité de la version 1 (voir « Complexité de la version 1 »)
3	Analyse du retour de la supervision (voir « Retour de la supervision »)
4	Étude d'une proposition de code d'un autre groupe : rejetée (voir « Proposition de code étudiée »)
5	Prototype complet (structures, entiers 16 bits, champs de bits) : mis de côté pour l'instant (voir « Pistes pour la version finale »)
6	**Version 2** : optimisation de la complexité et suppression des données redondantes, sur la base de la version 1 (voir « Version 2 optimisée »)

Retranscription en C (version 1)

Corrections du pseudocode faites lors de la traduction

- `Tab_Mesures[Nb_Capteurs - 1]` est trop petit d'une case : il faut `NB_CAPTEURS` cases (indices 0 à N−1). C'est pareil pour `Tab_Erreur` et `Tab_Moy_Instant`.
- `Decalage` écrit dans `Tableau[indice_max_Tableau]` et la moyenne divise par `indice_max_Tableau + 1`. Le tableau d'historique doit donc avoir `INDICE_MAX_TABLEAU + 1` cases.

- Les tableaux `Tab_Capteur_x` ont été regroupés dans un tableau à deux dimensions `Tab_Capteurs[N][T]`, pour pouvoir les parcourir avec une boucle.

Code de la version 1

La traduction du pseudo-code donne le programme suivant :

```
#include <Arduino.h>

// Constantes

#define CAPTEUR_TEMPERATURE 0
#define CAPTEUR_HUMIDITE    1
#define CAPTEUR_PRESSION    2
#define CAPTEUR_LUMIERE     3
#define CAPTEUR_SON         4

#define NB_CAPTEURS 5
#define NB_VAL 10


// Tableaux
float   Tab_Mesures[NB_CAPTEURS];
uint8_t Tab_Erreur[NB_CAPTEURS];
float   Tab_Moy_Instant[NB_CAPTEURS];
float   Tab_Capteurs[NB_CAPTEURS][NB_VAL];


// Prototypes des fonctions
uint8_t Lecture_capteur(float *Mesure, uint8_t i);
void Lecture_Capteurs(float *Mesures, uint8_t *Erreurs);
void Maj_Mesures(float *Mesures, uint8_t *Erreurs);
void Decalage(float *Tableau, float Valeur);
float Calcul_Moy_Instant(float *Tableau);
void Afficher(float *Tab_Moy, uint8_t *Erreurs);

void setup() {
    Serial.begin(9600);
    randomSeed(analogRead(A0));
}

void loop() {
    uint8_t i;

    Lecture_Capteurs(Tab_Mesures, Tab_Erreur);
    Maj_Mesures(Tab_Mesures, Tab_Erreur);
    for (i = 0; i < NB_CAPTEURS; i++)
        Tab_Moy_Instant[i] = Calcul_Moy_Instant(Tab_Capteurs[i]);

    Afficher(Tab_Moy_Instant, Tab_Erreur);
    delay(1000);
}


uint8_t Lecture_capteur(float *Mesure, uint8_t i) {
    *Mesure = random(1000) / 10.0 + i;
    return random(10) == 0;
}


void Lecture_Capteurs(float *Mesures, uint8_t *Erreurs) {
    float Mesure;
    uint8_t i;
    for (i = 0; i < NB_CAPTEURS; i++) {
        Erreurs[i] = 0;
        Mesure = 0;
        if (Lecture_capteur(&Mesure, i))
            Erreurs[i] = 1;
        else
            Mesures[i] = Mesure;
    }
```

```
65    }
66
67
68    void Maj_Mesures(float *Mesures, uint8_t *Erreurs) {
69        uint8_t i;
70        for (i = 0; i < NB_CAPTEURS; i++) {
71            if (Erreurs[i] != 1)
72                Decalage(Tab_Capteurs[i], Mesures[i]);
73        }
74    }
75
76
77    void Decalage(float *Tableau, float Valeur) {
78        uint8_t i;
79        for (i = 0; i < NB_VAL - 1; i++)
80            Tableau[i] = Tableau[i + 1];
81        Tableau[NB_VAL - 1] = Valeur;
82    }
83
84
85    float Calcul_Moy_Instant(float *Tableau) {
86        float Moyenne = 0;
87        uint8_t i;
88        for (i = 0; i < NB_VAL; i++)
89            Moyenne += Tableau[i];
90        return Moyenne / NB_VAL;
91    }
92
93
94    void Afficher(float *Tab_Moy, uint8_t *Erreurs) {
95        uint8_t i;
96        for (i = 0; i < NB_CAPTEURS; i++)
97            {
98            Serial.print(Tab_Moy[i], 1);
99            Serial.print(" ");
100           }
101       Serial.print("| err: ");
102       for (i = 0; i < NB_CAPTEURS; i++)
103           {
104           Serial.print(Erreurs[i]);
105           Serial.print(" ");
106           }
107       Serial.println();
108   }
```

Complexité de la version 1

Paramètres

On exprime le coût en fonction de deux paramètres :

- **N** = nombre de capteurs (`NB_CAPTEURS`), ici **4** ;
- **T** = taille de l'historique (`TAILLE_TABLEAU = INDICE_MAX_TABLEAU + 1`), ici **10**.

Méthode de calcul

1. **Choisir les opérations élémentaires.** On compte les affectations, les opérations arithmétiques, les comparaisons sur les données et les appels de lecture capteur. Chacune coûte 1. La gestion des compteurs de boucle (`i++`, `i < T`) n'est pas comptée, et l'affichage série non plus.
2. **Compter chaque fonction, de l'intérieur vers l'extérieur :**
   - une **séquence** d'instructions : on **additionne** leurs coûts ;
   - une **boucle** de k tours : on **multiplie** le coût du corps par k ;

- un **appel de fonction** : on remplace l'appel par le coût de la fonction.

3. **Additionner** le coût des fonctions appelées dans un tour de `loop()`.
4. **Passer à la notation O** : on garde le terme qui grandit le plus vite et on enlève les constantes.

Calcul fonction par fonction

`Decalage(tableau, valeur)`

```
for (int i = 0; i < INDICE_MAX_TABLEAU; i++)   // T - 1 tours
    tableau[i] = tableau[i + 1];                // 1 affectation
tableau[INDICE_MAX_TABLEAU] = valeur;           // 1 affectation
```

Coût : (T − 1) × 1 + 1 = **T**

`Calcul_Moy_Instant(tableau)`

```
float moyenne = 0;                              // 1
for (int i = 0; i < TAILLE_TABLEAU; i++)        // T tours
    moyenne += tableau[i];                      // 1 addition
return moyenne / TAILLE_TABLEAU;                // 1 division
```

Coût : 1 + T + 1 = **T + 2**

`Lecture_Capteurs()` : une boucle de N tours, avec dans chaque tour :

`Tab_Erreur = false` (1), lecture du capteur (1), test (1), affectation de `Tab_Mesures` ou de `Tab_Erreur` (1), soit 4.

Coût : **4N**

`Maj_Mesures()` : une boucle de N tours, avec dans chaque tour un test (1) et un appel à `Decalage` (T).

Coût : **N(T + 1)**

Calcul des moyennes dans `loop()` : une boucle de N tours, avec dans chaque tour un appel à `Calcul_Moy_Instant` (T + 2) et l'affectation dans `Tab_Moy_Instant` (1).

Coût : **N(T + 3)**

Total pour un tour de loop()

$$C_1(N, T) = 4N + N(T + 1) + N(T + 3)$$
$$= N(2T + 8)$$

Valeur avec N = 4 et T = 10 :

$$C_1 = 4 \times (2 \times 10 + 8) = 4 \times 28 = 112 \text{ opérations par tour de loop()}$$

Complexité : O(N·T). Le coût est proportionnel au nombre de capteurs multiplié par la taille de l'historique. Si on double T (moyenne sur 20 mesures), on double presque le temps de calcul.

Remarques :

- **Pire cas** : tous les capteurs répondent, donc tous les décalages sont faits. C'est le cas calculé ci-dessus. Si un capteur est en erreur, son `Decalage` est sauté, mais l'ordre de grandeur reste le même.
- `setup()` coûte aussi O(N·T) (mise à zéro des tableaux), mais une seule fois au démarrage, donc on ne le compte pas dans le coût de fonctionnement.
- N et T sont des constantes fixées à la compilation. Sur la carte, le coût d'un tour est donc fixe. La notation O(N·T) indique comment il évolue si on modifie ces paramètres, ce qui compte parce que le système est censé être évolutif.

Complexité en mémoire

Sur Arduino (AVR) : `float` = 4 octets, `bool` = 1 octet, `int` = 2 octets.

Donnée	Taille	Octets (N = 4, T = 10)
`Tab_Mesures[N]`	4N	16
`Tab_Erreur[N]`	N	4
`Tab_Moy_Instant[N]`	4N	16
`Tab_Capteurs[N][T]`	4NT	160
Total	**N(4T + 9)**	**196 octets**

Complexité en mémoire : O(N·T)

Problèmes identifiés

1. **`Decalage` recopie tout l'historique** à chaque nouvelle mesure (T copies), alors qu'une seule valeur change.
2. **`Calcul_Moy_Instant` refait toute la somme** à chaque tour (T additions), alors que la somme ne change que d'une valeur entrante et d'une valeur sortante.
3. **Des données sont stockées en double :**
   - `Tab_Mesures` sert seulement d'intermédiaire entre la lecture et `Maj_Mesures`. La valeur est ensuite recopiée dans l'historique, donc elle existe deux fois ;
   - `Tab_Moy_Instant` contient une valeur qu'on peut recalculer directement à partir de la somme de l'historique.

Proposition de code étudiée (rejetée)

Une proposition de code (5 capteurs, tableaux `tab_C1` à `tab_C5`, `tab_er_C1` à `tab_er_C5`) a été analysée. Elle contient deux bonnes idées :

- revenir à l'indice 0 en fin de tableau : c'est le principe du buffer circulaire ;
- un tableau de pointeurs vers les historiques, pour éviter les copies.

Mais elle ne répond à aucune critique de la supervision et ne compile pas :

- elle ajoute des tableaux au lieu de les remplacer par des structures ;
- les erreurs sont stockées sur 50 `int` au lieu d'1 bit par capteur ;
- elle utilise encore `float`, avec une division entière qui tronque ;
- elle contient des erreurs de C : tableaux `[4]` initialisés avec 5 éléments, type `str` inexistant, opérateur virgule dans `(a, b, c…)/5`, `return` en dehors de la fonction, indices `i+6` hors du tableau, `si i=9` (affectation au lieu de comparaison) ;

- la boucle `i` est remise à 0 à l'intérieur d'elle-même, ce qui crée une boucle infinie ;
- `j` va de 1 à 4, donc seulement 4 capteurs sur 5 sont traités.

Version 2 optimisée

On part de la version 1 et on change **seulement** la complexité et la redondance des données. Les types (`float`) et l'organisation en tableaux restent les mêmes.

Changements

a) Buffer circulaire au lieu du décalage

Au lieu de décaler tout le tableau, on garde pour chaque capteur l'indice de la case la plus ancienne (`Tab_Index`). La nouvelle mesure écrase cette case, puis l'indice avance d'une case et revient à 0 après la dernière.

L'ordre des valeurs dans le tableau n'a pas d'importance, puisqu'on ne s'en sert que pour la moyenne.

b) Somme glissante

On garde la somme de l'historique (`Tab_Somme`). À chaque nouvelle mesure :

$$\text{somme} = \text{somme} + \text{nouvelle_valeur} - \text{valeur_écrasée}$$

La moyenne devient alors une simple division : `Tab_Somme[c] / TAILLE_TABLEAU`.

c) Suppression des redondances

Donnée supprimée	Pourquoi
`Tab_Mesures`	La mesure est écrite directement dans l'historique. La lecture et la mise à jour se font dans une seule boucle (`Lecture_Capteurs`), donc `Maj_Mesures` disparaît aussi.
`Tab_Moy_Instant`	La moyenne se calcule en O(1) depuis `Tab_Somme`, il n'y a plus besoin de la stocker.

`Tab_Erreur` est conservé, car c'est une information qui n'existe nulle part ailleurs.

Code de la version 2

Ces changements donnent le programme suivant :

```
#include <Arduino.h>


// Configuration

// Un #define par capteur : num_capteur allant de 0 a NB_CAPTEURS - 1
#define CAPTEUR_TEMPERATURE 0
#define CAPTEUR_HUMIDITE    1
#define CAPTEUR_PRESSION    2
#define CAPTEUR_LUMIERE     3

#define NB_CAPTEURS 4

// Nombre de mesures utilisees pour la moyenne instantanee = INDICE_MAX_TABLEAU + 1
#define INDICE_MAX_TABLEAU 9
#define TAILLE_TABLEAU (INDICE_MAX_TABLEAU + 1)

bool  Tab_Erreur[NB_CAPTEURS];

// Un tableau d'historique par capteur (Tab_Capteur_num_capteur), utilise en buffer circulaire
float Tab_Capteurs[NB_CAPTEURS][TAILLE_TABLEAU];
```

```
int   Tab_Index[NB_CAPTEURS];   // case qui contient la plus ancienne mesure
float Tab_Somme[NB_CAPTEURS];   // somme des mesures de l'historique


// Lecture materielle d'un capteur (a remplacer par les vrais drivers)
// Retourne false si le capteur est en erreur.
bool Lire_Capteur(int num_capteur, float *mesure)
{
    switch (num_capteur) {
        case CAPTEUR_TEMPERATURE: *mesure = 20.0f; return true;
        case CAPTEUR_HUMIDITE:    *mesure = 50.0f; return true;
        case CAPTEUR_PRESSION:    *mesure = 1013.0f; return true;
        case CAPTEUR_LUMIERE:     *mesure = analogRead(A0); return true;
        default:                  return false;
    }
}


// Fonctions factorisees


// Remplace la plus ancienne mesure par la nouvelle et met la somme a jour
void Decalage(int num_capteur, float valeur)
{
    int i = Tab_Index[num_capteur];

    Tab_Somme[num_capteur] += valeur - Tab_Capteurs[num_capteur][i];
    Tab_Capteurs[num_capteur][i] = valeur;

    i++;
    if (i == TAILLE_TABLEAU) {
        i = 0;
    }
    Tab_Index[num_capteur] = i;
}

float Calcul_Moy_Instant(int num_capteur)
{
    return Tab_Somme[num_capteur] / TAILLE_TABLEAU;
}


// Procedures principales


// Lit chaque capteur et met directement son historique a jour
void Lecture_Capteurs()
{
    for (int num_capteur = 0; num_capteur < NB_CAPTEURS; num_capteur++) {
        float mesure;
        Tab_Erreur[num_capteur] = false;

        if (Lire_Capteur(num_capteur, &mesure)) {
            Decalage(num_capteur, mesure);
        } else {
            Tab_Erreur[num_capteur] = true;
        }
    }
}


// Arduino


void setup()
{
    Serial.begin(9600);

    for (int c = 0; c < NB_CAPTEURS; c++) {
        Tab_Erreur[c] = false;
        Tab_Index[c] = 0;
        Tab_Somme[c] = 0;
        for (int i = 0; i < TAILLE_TABLEAU; i++) {
```

```
                Tab_Capteurs[c][i] = 0;
            }
        }
    }

    void loop()
    {
        Lecture_Capteurs();

        for (int num_capteur = 0; num_capteur < NB_CAPTEURS; num_capteur++) {
            Serial.print("Capteur ");
            Serial.print(num_capteur);
            Serial.print(" : moyenne = ");
            Serial.println(Calcul_Moy_Instant(num_capteur));
        }

        delay(1000);
    }
```

Complexité de la version 2

On compte de la même façon que pour la complexité de la version 1.

`Decalage(num_capteur, valeur)` : lire l'indice (1), mettre à jour la somme (2 : une soustraction et une addition), écrire la valeur (1), incrémenter l'indice (1), le tester (1), l'enregistrer (1).

Coût : **7**, quel que soit T

`Calcul_Moy_Instant(num_capteur)` : une division.

Coût : **1**

`Lecture_Capteurs()` : une boucle de N tours, avec dans chaque tour `Tab_Erreur = false` (1), la lecture (1), le test (1) et l'appel à `Decalage` (7), soit 10.

Coût : **10N**

Calcul des moyennes dans `loop()` : N × 1 = **N**

$$C_2(N) = 10N + N = 11N$$

Avec N = 4 : **C_2 = 44 opérations par tour**, contre 112 pour la version 1.

Complexité : O(N). Le coût ne dépend plus de T.

T (taille de l'historique)	Version 1 : N(2T + 8)	Version 2 : 11N	Gain
10	112	44	≈ 2,5×
50	432	44	≈ 10×
100	832	44	≈ 19×

Mémoire de la version 2

Donnée	Taille	Octets (N = 4, T = 10)
`Tab_Erreur[N]`	N	4
`Tab_Capteurs[N][T]`	4NT	160
`Tab_Index[N]`	2N	8
`Tab_Somme[N]`	4N	16
Total	**N(4T + 7)**	**188 octets**

La mémoire baisse un peu (196 → 188 octets) : on a supprimé deux tableaux redondants, mais ajouté l'indice et la somme. Ça reste en **O(N·T)**, parce que l'historique est indispensable pour connaître la valeur qui sort de la moyenne.

Le gain principal de la version 2 est donc sur le **temps**. Pour gagner vraiment de la mémoire, il faut changer les types (voir « Pistes pour la version finale »).

Vérification

Le programme a été compilé et exécuté sur PC, avec une fausse bibliothèque Arduino qui renvoie des valeurs fixes. Les moyennes affichées sont celles qu'on attend de la version 1 : elles montent progressivement pendant les 10 premiers tours (l'historique part de 0), puis se stabilisent à la valeur mesurée.

Point d'attention : avec des `float`, la somme glissante peut accumuler de petites erreurs d'arrondi sur un très grand nombre de mesures à virgule. Ce problème disparaît avec des entiers (voir « Pistes pour la version finale »).

Pistes pour la version finale

Un prototype a été écrit pour répondre aux autres remarques de la supervision. Il est mis de côté pour l'instant :

Remarque de la supervision	Piste
Remplacer les tableaux par des structures	Une `struct Capteur` qui regroupe l'historique, la somme, l'indice et l'erreur d'un capteur. On passe un **pointeur** sur la structure, donc il n'y a aucune copie.
Mémoire limitée	Des **champs de bits** : `erreur : 1` occupe vraiment 1 bit, `index : 3` occupe 3 bits.
Pas besoin de réels	Des entiers `int16_t`, éventuellement en virgule fixe (23,5 °C stocké comme 235). Ça divise par deux la mémoire de l'historique, et les calculs sont beaucoup plus rapides sur un AVR qui n'a pas d'unité de calcul flottant.
Temps minimal	Une taille d'historique en **puissance de 2** (8 ou 16) : la division devient un décalage de bits et le retour à 0 de l'indice se fait tout seul.

Estimation pour N = 4 et T = 8 : environ **80 octets**, contre 196 pour la version 1, avec un coût en **O(N)**.